

Flow of Control

27 May 2026 18:37

Selection:

1. `if condition:`
 `statement(s)`

2. `if condition:`
 `statement(s)`
 `else:`
 `statement(s)`

3. `if condition:`
 `statement(s)`
 `elif condition:`
 `statement(s)`
 `elif condition:`
 `statement(s)`
 :
 :
 `else:`
 `statement`

Nested:

4. `if condition:`
 `if condition:`
 :
 `elif condition:`
 `statement:`
 `if condition:`

Any level of nesting is required.
we can do.

`match (variable):`
 `case value:`
 `statements`
 `case value:`
 `statement(s)`
 :
 :
 `case _:`
 `statement(s)`

`variable == value`,
that only case will be
executed.

} when all cases fails then '-' case
will execute.

```
num1 = int(input('Enter first number: '))
num2 = int(input('Enter second number: '))
op = input('Enter operator (+,-,*,/): ')
```

```
match op:
```

```
    case '+':
        print(f'Sum = {num1 + num2}')
```

```
    case '-':
        print(f'Difference = {num1 - num2}')
```

```
    case '*':
        print(f'Product = {num1 * num2}')
```

```
    case '/':
        print(f'Quotient = {num1 / num2}')
```

```
    if num2 == 0:
```

```

case '/': formatted thing
    if num2 == 0:
        print('Division by 0 is not allowed')
    else:
        print(f'Quotient = {num1 / num2}')
case _:
    print('Wrong operator selected')

```

```

>>> price = 1234.5678
>>> print(f'Price: {price:.2f}')
Price: 1234.57
>>> print(f'Price: {price:.3f}')
Price: 1234.568
>>> print(f'Price: {price:,.3f}')
Price: 1,234.568

```

number of digits after decimal place.

comma as thousand separator

```

score = int(input('Enter your score: '))
print(f"Result: {'Pass' if score >= 50 else 'Fail'}")

```

Format	Example	Output
Decimal integer	f"{42:d}"	42
Leading zeros	f"{42:05d}"	00042
Leading spaces	f"{42: 5d}"	42
Fixed-point	f"{3.14159:.2f}"	3.14
Comma separator	f"{100000:,}"	1,000,000
Binary	f"{10:b}"	1010

For example:

```

score = int(input('Enter your score: '))
print(f"Result: {'Pass' if score >= 50 else 'Fail'}")
print(f"{score:03d}")
print(f"{score:3d}")
print(f"{1000000:,}")
print(f"{score:b}")

```

Output:

```

Enter your score: 10
Result: Fail
010
10
10,000,000
1010

```

Repetition :

```

print(1)
print(2)

```

```
print(3)
print(4)
```

while condition:
 Statement(s)

For example:
 I want to print values from 1 to 10.

```
start = 1
stop = 10
while start <= stop:
    print(start)
    start += 1
```

```
start = int(input('Enter the start value: '))
stop = int(input('Enter the stop value: '))
while start <= stop: ← when this condition is false then else block will execute.
    print(start)
    start += 1
    if start == stop: #if this condition is true then else part of while loop
will not work
        break
else: ← not compulsory, if required use it.
    print('Loop ends')
```

For loop:
for var in range(stop):
 Statement(s)

The value of stop in range() indicates the loop will execute from 0 to stop-1.

```
>>> for var in range(5):
```

```
...     print(var) ↑ stop
```

```
...
```

```
0
```

```
1
```

```
2
```

```
3
```

```
4
```

```
>>> for var in range(5):
```

```
...     print('Hello')
```

```
...     print('Good')
```

```
...
```

```
Hello
```

```
Good
```

```
Hello
```

```
Good
```

Hello
Good
Hello
Good
Hello
Good

Another syntax of for loop:

```
for var in range(start, stop):  
    Statement(s)
```

This for loop will execute from start to stop-1. It creates a usable list of values from start to stop-1.

Example:

```
>>> for var in range(10, 20):  
...     if var % 2 == 0 or var % 3 == 0:  
...         print(var)  
...  
10  
12  
14  
15  
16  
18
```

Another syntax of for loop:

```
for var in range(start, stop, skip):  
    statement(s)
```

This for loop will execute from start to stop - 1 by incrementing the value of stop by skip. It creates a usable list of values from start to stop - 1 and values present in the list are created by adding the value of skip with previous values.

Example:

```
>>> for var in range(1, 50, 3):  
...     print(var)  
...  
1  
4  
7  
10  
13  
16  
19  
22  
25  
28  
31  
34  
37  
40
```

43
46
49

Reverse ordering:

```
>>> for var in range(50, 1, -3):  
...     print(var, end = ' ')  
...  
50 47 44 41 38 35 32 29 26 23 20 17 14 11 8 5 2
```

```
vowels='aeiouAEIOU'  
name = input('Enter any name: ')  
count = 0  
for char in name:  
    if char in vowels:  
        count += 1  
print(f'Number of vowels: {count}')
```

Output:

Enter any name: Monhandas
Number of vowels: 3

Another example:

```
import random as R  
li = [item for item in range(R.randint(1,10), R.randint(20, 40), R.randint(1, 6))]  
countE = 0  
countO = 0  
for var in li:  
    if var % 2 == 0:  
        countE += 1  
    else:  
        countO += 1  
    print(var, end=' ')  
  
print(f'\nCount of even numbers: {countE}')
```

Handwritten annotations for the range function in the list comprehension:
- *module* points to `random`
- *alias* points to `as R`
- *start* points to `R.randint(1,10)`
- *stop* points to `R.randint(20, 40)`
- *skip* points to `R.randint(1, 6)`

```
print(f'Count of odd numbers: {countO}')
```

Output:

```
PS C:\Users\bluej\Documents\Students\AaravTibrewal\ForGit\Coding> & C:/Python314/py  
ments/Students/AaravTibrewal/ForGit/Coding/Chapter1/forLoop2.py  
8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34  
Count of even numbers: 14  
Count of odd numbers: 13  
PS C:\Users\bluej\Documents\Students\AaravTibrewal\ForGit\Coding> & C:/Python314/py  
ments/Students/AaravTibrewal/ForGit/Coding/Chapter1/forLoop2.py  
9 12 15 18 21 24  
Count of even numbers: 3  
Count of odd numbers: 3
```

start = 8
stop = 35
skip = 1

start = 9 stop = 25 skip = 3

WAP to generate the sequence: -5, 10, -15, 20, -25,... up to n, where n is an integer input by the user.

WAP to input number of rows and print the following pattern.

N = 5

```
    1
  2 1 2
3 2 1 2 3
4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
```

N = 5

```
5 4 3 2 1 2 3 4 5
 4 3 2 1 2 3 4
   3 2 1 2 3
    2 1 2
     1
```